



Hybrid Energy-LQR Control of Rotary Inverted Pendulum (RIP)

ผู้จัดทำ

นางสาวดิษย์ธร	สุทธาเวช	รหัสนักศึกษา 66340500019
นางสาวบุญยาพร	ปรีชาศุทธิ์	รหัสนักศึกษา 66340500031
นายภคิน	บุญชนะชัย	รหัสนักศึกษา 66340500037
นายภูวนวัฒน์	บุญเกิด	รหัสนักศึกษา 66340500042

รายวิชา FRA333 Kinematics of Robotics System

สถาบันวิทยาการหุ่นยนต์ภาคสนาม มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ประจำปีการศึกษา 1/2568

สารบัญ

รายการ	หน้า
ที่มาและความสำคัญ.....	1
วัตถุประสงค์	1
ขอบเขตการทำงาน.....	2
โครงสร้างทางกล	3
Kinematics.....	7
1. Forward Pose Kinematics.....	7
2. Forward Velocity Kinematics (FVK)	9
3. Inverse Jacobian Matrix	9
4. Dynamics Equation.....	10
State Space ของระบบ	12
Energy-Based Swingup.....	15
Linear Quadratic Regulator.....	17
Modelling (URDF)	19
ROS Implementation.....	21
1. Hardware Interface – micro-ROS (STM32).....	21
2. Robot Description & TF – invpendulum_description.....	22
3. Simulation & Comparison – invpendulum_simulation.....	22
4. Visualization – RViz2.....	23
สรุปและวิเคราะห์ผลการทดลอง	24

ที่มาและความสำคัญ

Rotary Inverted Pendulum (RIP) เป็นระบบตัวอย่างมาตรฐาน (Canonical System) ที่มีบทบาทสำคัญในงานวิจัยด้านทฤษฎีระบบควบคุม (Control Theory) เนื่องจากเป็นระบบที่มีคุณสมบัติ Nonlinear, Underactuated, และไม่เสถียรโดยธรรมชาติ โดยเฉพาะอย่างยิ่งในแง่ของการควบคุมพฤติกรรมที่ไม่เสถียร ซึ่งทำให้เป็นกรณีศึกษาที่ทำนายในการทดสอบและพัฒนาเทคนิคการควบคุมระบบที่มีความซับซ้อนสูง

RIP เป็นระบบที่มีการเคลื่อนไหว 2 ลักษณะหลัก คือ การแกว่งขึ้น (Swing-up) ซึ่งเป็นการเหวี่ยงลูกตุ้มจากตำแหน่งห้อยลงไปสู่ตำแหน่งที่ตั้งตรง และการรักษาสถิต (Stabilization) ซึ่งเป็นการควบคุมให้ลูกตุ้มอยู่ในตำแหน่งที่ตั้งตรงอย่างเสถียรและไม่ล้ม โดยการควบคุมในช่วง Swing-up เป็นส่วนที่ทำนาย เนื่องจากพฤติกรรมของระบบในช่วงนี้มีความไม่เป็นเชิงเส้นอย่างมาก ทำให้ระบบควบคุมเชิงเส้น (Linear Control) แบบทั่วไปไม่สามารถทำงานได้อย่างมีประสิทธิภาพ ดังนั้นเพื่อให้สามารถควบคุมระบบได้อย่างมีประสิทธิภาพ จึงควบคุม Swing-up ด้วย Energy-based Control ที่เน้นการเพิ่มพลังงานรวมของระบบ เพื่อให้เข้าสู่บริเวณที่เหมาะสมสำหรับการใช้งาน Linear Quadratic Regulator (LQR) ในการควบคุมและรักษาสถิตในช่วงหลังจากนั้น การควบคุมในรูปแบบนี้ช่วยให้สามารถใช้ LQR ในการรักษาสถิตของลูกตุ้มในตำแหน่งที่ตั้งตรงได้อย่างมีประสิทธิภาพ

เพื่อรองรับการพัฒนาาระบบควบคุมดังกล่าว คณะผู้จัดทำจึงได้ทำการสร้างแบบจำลอง RIP โดยใช้ Lagrange Method และแปลงเป็น State-space Model ซึ่งเป็นแบบจำลองเชิงสถานะที่ใช้ในการควบคุมระบบ หลังจากนั้นได้พัฒนาระบบ ROS2 สำหรับการสื่อสารข้อมูล การประมวลผลข้อมูลเชิง Kinematics ในทางทฤษฎี และการแสดงผลข้อมูลในรูปแบบ Real-time ผ่าน Rviz ซึ่งช่วยให้ผู้ใช้สามารถมองเห็นพฤติกรรมของระบบที่เกิดขึ้นจริงจาก Hardware และการคำนวณในทางทฤษฎีได้อย่างชัดเจน

วัตถุประสงค์

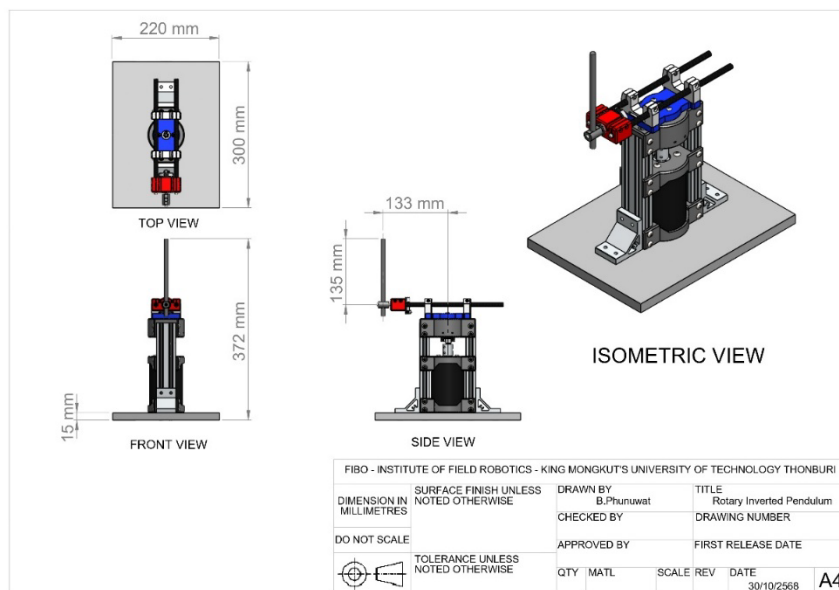
- เพื่อพัฒนาและออกแบบระบบควบคุมแบบผสมผสาน (Hybrid Control System) สำหรับ Rotary Inverted Pendulum ที่สามารถทำการควบคุมได้ทั้งช่วงการแกว่งขึ้น (Swing-up) และการรักษาสถิต (Stabilization)
- เพื่อศึกษาการใช้แนวคิด Energy-based Control สำหรับการเพิ่มพลังงานรวมของระบบ (Total Energy) เพื่อให้ Pendulum แกว่งขึ้นสู่จุดอ้างอิง ก่อนส่งต่อให้ Linear Quadratic Regulator (LQR) ทำหน้าที่ควบคุมให้ Pendulum คงอยู่ในตำแหน่งที่ตั้งตรง
- เพื่อสร้างแบบจำลองทางคณิตศาสตร์ของระบบโดยใช้ Lagrange Method และแปลงเป็นสมการเชิงสถานะ (State-space Model) เพื่อใช้ในการออกแบบตัวควบคุม LQR
- เพื่อพัฒนาระบบสื่อสารบน ROS2 โดยให้ NUCLEO-G474RE ทำหน้าที่ประมวลผล Control Loop และ Publish สถานะข้อต่อ (Joint States)
- เพื่อพัฒนา Node สำหรับรับสถานะข้อต่อเพื่อคำนวณ Forward Position Kinematics (FPK), Forward Velocity Kinematics (FVK) แบบ Real-time
- เพื่อแสดงผลการเคลื่อนไหว (Dynamic Motion) ของระบบแบบ Real-time ผ่าน RViz ด้วยแบบจำลอง URDF

- เพื่อออกแบบและทดสอบระบบควบคุมบน Microcontroller NUCLEO-G474RE ซึ่งมีความสามารถในการประมวลผลแบบ Real-time
- เพื่อประยุกต์ใช้ความรู้จากวิชา FRA333 Kinematics For Robotics

ขอบเขตการทำงาน

- โครงการนี้ออกแบบและสร้าง Rotary Inverted Pendulum (RIP) ที่ขับเคลื่อนด้วย Brushed DC Motor ซึ่งทำการควบคุมด้วยสัญญาณแรงดันไฟฟ้า
- ระบบจะใช้ Energy-based Controller ในการแกว่ง Pendulum ขึ้น และใช้ LQR Controller สำหรับการทำให้สมดุลในตำแหน่งที่ตั้งตรง
- ระบบจะใช้ AMT103-V Encoder สำหรับตรวจวัดมุมของ Rotary Arm และมุมของ Pendulum เพื่อใช้เป็น Feedback ของระบบควบคุม
- ระบบจะใช้ Cytron MD20A Motor Driver ในการขับมอเตอร์ โดยรับสัญญาณ PWM และ Direction จากไมโครคอนโทรลเลอร์
- ระบบจะใช้ NUCLEO-G474RE ในการประมวลผลหลัก โดยทำงานแบบ Real-time บน Core M4 และสื่อสารกับ Sensor และ Driver ผ่าน Peripheral Interface
- การแสดงผลการเคลื่อนไหวของหุ่นยนต์จะใช้ RViz ร่วมกับแบบจำลอง URDF (Unified Robot Description Format)

โครงสร้างทางกล

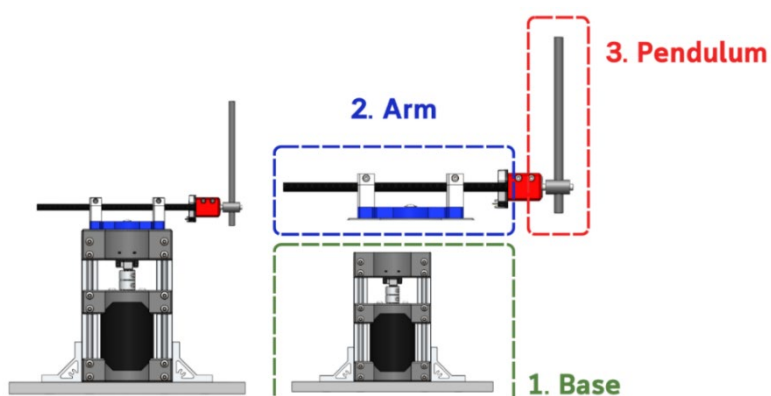


รูปที่ 1 Drawing โครงสร้างทางกลของระบบ RIP

โครงสร้างทางกลของระบบ Rotary Inverted Pendulum (RIP) ได้รับการออกแบบเพื่อให้มีความแข็งแรง น้ำหนักเบา และมีความแม่นยำในการเคลื่อนที่เชิงไดนามิกสูงสุด โดยใช้ชิ้นส่วนมาตรฐาน (Standard Parts) เป็นองค์ประกอบหลัก เช่น อะลูมิเนียมโปรไฟล์ แบร็ก และเพลาอะลูมิเนียม ประกอบร่วมกับชิ้นส่วนที่ขึ้นรูปด้วย 3D Printing เฉพาะในตำแหน่งที่ต้องการรูปทรงพิเศษหรือมีความจำเป็นในการลดน้ำหนักและปรับแต่งให้เหมาะสม

ขนาดของโครงสร้างโดยรวม

- ขนาดฐาน (Top View) : 220 × 300 mm
- ความสูงรวมของระบบ (Front View) : 372 mm



รูปที่ 2 ส่วนประกอบหลักของระบบ RIP

โครงสร้างของระบบแบ่งออกเป็น 3 ส่วนหลัก ได้แก่

1. ฐาน (Base)

ฐานทำหน้าที่เป็นโครงสร้างรองรับน้ำหนักรวมของระบบและเป็นจุดอ้างอิงของโครงสร้างเชิงพิกัด (Reference Frame) ในการสร้างแบบจำลอง ฐานประกอบด้วยอะลูมิเนียมโปรไฟล์และแผ่นยึดที่ติดตั้งมอเตอร์ ชุด Encoder และอุปกรณ์อิเล็กทรอนิกส์อื่น ๆ โดยการออกแบบได้เน้นให้ฐานมีมวลและความแข็งแรงเพียงพอ ซึ่งมีความสำคัญต่อการลดแรงสั่นสะเทือนที่เกิดขึ้นจากแรงบิดของมอเตอร์และการเหวี่ยงตัว Pendulum ในระหว่างการทำงานของระบบจริง

2. แขนหมุน (Arm)

แขนหมุนเป็นชิ้นส่วนที่เชื่อมต่อโดยตรงกับแกนมอเตอร์ และเป็นจุดหมุนแรกของระบบ (Rotary Joint) มีหน้าที่ส่งแรงบิดเพื่อเหวี่ยง Pendulum เข้าสู่สภาวะ Swing-up รวมถึงใช้ในกระบวนการควบคุมการทรงตัวเชิงเส้นและเชิงมุมของ Pendulum

รายละเอียดของแขนหมุน (Arm)

- ผลิตจากชิ้นส่วนอะลูมิเนียมและชิ้นส่วน 3D Printing เพื่อให้แข็งแรงและน้ำหนักเบา
- ติดตั้ง AMT103-V Encoder สำหรับวัดมุมของแขนหมุน (θ)
- ความยาวแขนหมุนจากแกนมอเตอร์ถึงจุดหมุน Pendulum : $L_1 = 133 \text{ mm}$

โครงสร้างแขนหมุน (Arm) ถูกออกแบบให้มีโมเมนต์ความเฉื่อยต่ำ เพื่อให้ตอบสนองต่อแรงบิดจากมอเตอร์ได้รวดเร็วและลดความล่าช้าของระบบควบคุม

3. ลูกตุ้ม (Pendulum)

ลูกตุ้มเป็นชิ้นส่วนที่ต้องการควบคุมให้รักษาสมดุลในตำแหน่งกลับหัว ซึ่งเป็นสภาวะไม่เสถียรโดยธรรมชาติ (Unstable Equilibrium)

รายละเอียดของลูกตุ้ม (Pendulum)

- ประกอบด้วยเพลาลูกตุ้มและชิ้นส่วนจับยึดที่ออกแบบให้มีการกระจายมวลเหมาะสม
- ติดตั้ง AMT103-V Encoder เพื่อวัดมุมของลูกตุ้ม (ϕ หรือ β)
- ความยาวลูกตุ้มจากจุดหมุนถึงปลาย : $L_2 = 135 \text{ mm}$

การกำหนดตำแหน่งและการกระจายมวลอย่างถูกต้องมีผลโดยตรงต่อสมการโมเดลเชิงไดนามิกของระบบ รวมถึงเสถียรภาพในการควบคุมแบบ LQR

การวิเคราะห์คุณสมบัติมวล (Analyzed Mass Properties)

ในการสร้างแบบจำลองเชิงไดนามิก ค่ามวล (Mass), จุดศูนย์กลางมวล (Center of Mass) และโมเมนต์ความเฉื่อย (Moments of Inertia) เป็นตัวแปรสำคัญโดยตรงต่อสมการการเคลื่อนที่ที่ได้จาก Lagrangian Method ดังนั้นจึงต้องใช้ข้อมูลมวลที่เป็น Ground Truth เพื่อให้แบบจำลองมีความแม่นยำเมื่อเปรียบเทียบกับระบบจริง

เนื่องจากระบบ RIP มีชิ้นส่วนที่เคลื่อนที่หลัก 2 ส่วน ได้แก่ แขนหมุน (Arm) และลูกตุ้ม (Pendulum) จึงทำการวิเคราะห์คุณสมบัติมวลเฉพาะชิ้นส่วนที่มีการเคลื่อนที่ (Moving Parts) เท่านั้น

1. การชั่งน้ำหนักชิ้นส่วน (Measured Mass Data)

หลังจากประกอบชิ้นส่วนจริงเรียบร้อยแล้ว ได้ทำการแยก Arm และ Pendulum ออกมาชั่งน้ำหนักจริงทีละชิ้นด้วยเครื่องชั่งน้ำหนักดิจิทัล เพื่อให้ค่ามวลที่ได้สามารถนำไปใช้คำนวณ Center of Mass และ Moments of Inertia ได้อย่างถูกต้อง โดยน้ำหนักของแต่ละชิ้นส่วนเป็นไปดังตารางนี้

ส่วนประกอบ	ชื่อชิ้นส่วน	ค่าน้ำหนัก (g)
แขนหมุน (Arm)	Base Swing Arm	25.5
	Bearing Holder + Ball Bearing	86.5
	Carbon Fiber Arm Shaft * 2	58
	Shaft Support Block * 2	24
	Pendulum Shaft Housing + Ball Bearing	33.5
	AMT103-V Encoder	16
ลูกตุ้ม (Pendulum)	Pendulum Rotating Shaft	28
	Pendulum Shaft	62

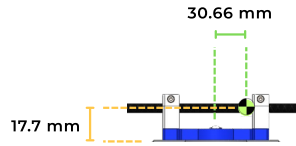
2. การวิเคราะห์ Center of Mass (CoM) และ Moments of Inertia (Iol)

หลังจากได้ค่ามวลจริงแล้ว ได้นำข้อมูลดังกล่าวไปกำหนดเป็น Custom Mass Properties ในโปรแกรม SolidWorks เพื่อให้การคำนวณ Center of Mass และ Moments of Inertia สอดคล้องกับน้ำหนักจริงของระบบ โดยไม่ใช้ค่า Density เริ่มต้นของวัสดุ ซึ่งอาจคลาดเคลื่อนจากของจริง

- Center of Mass (CoM): เป็นตำแหน่งของจุดศูนย์กลางมวลของชิ้นส่วนแต่ละชิ้น วัดในหน่วยมิลลิเมตร โดยอิงจากชุดแกนพิกัดเดียวกับ Assembly
- Moments of Inertia (Iol): วัดเป็นหน่วย mm² ตามแกน พร้อมค่า Principal Moment และค่า Orientation ของแกนเฉื่อยหลัก โดยค่าที่ได้ถูกนำไปใช้ดังนี้
 - การหาสมการไดนามิกของระบบ

- การสร้างแบบจำลองเชิงตัวเลข (Simulation)
- การเปรียบเทียบ Real Model vs Theoretical Model ใน ROS2 และ Rviz

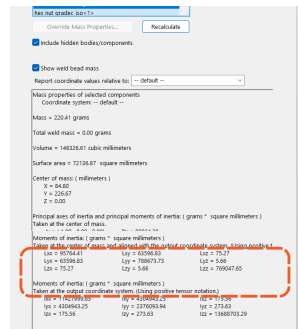
CENTER OF MASS :



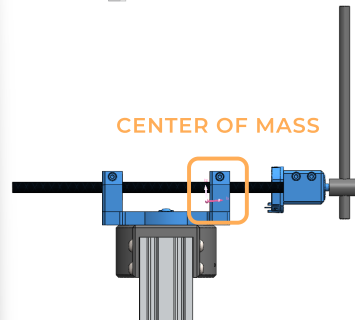
Center of Mass : 35.402 mm

MOMENTS OF INERTIA :

769047.65 (mm)²

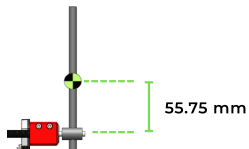


Arm



รูปที่ 3 การวิเคราะห์คุณสมบัติมวลของ Arm

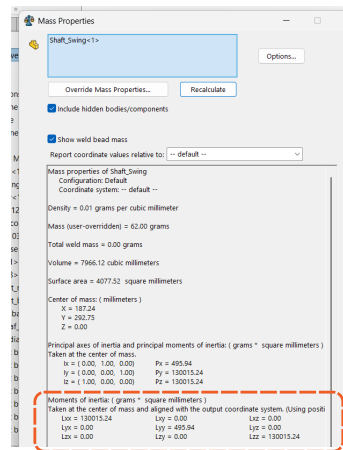
CENTER OF MASS :



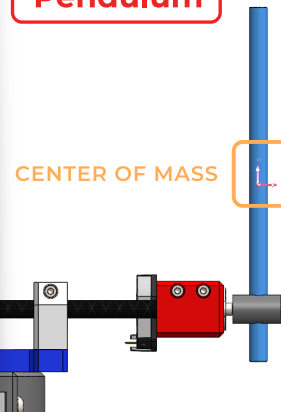
Center of Mass : 55.75 mm

MOMENTS OF INERTIA :

130015.24 (mm)²



Pendulum



รูปที่ 4 การวิเคราะห์คุณสมบัติมวลของ Pendulum

ตารางสรุปผลการวิเคราะห์ Center of Mass และ Moments of Inertia

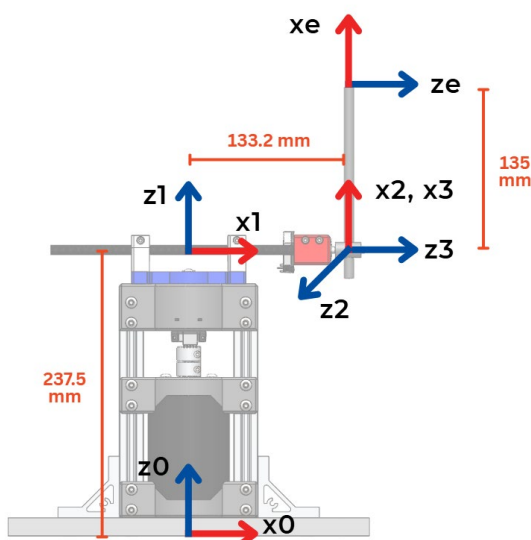
ส่วนประกอบ	Center of Mass (mm)	Moments of Inertia (g · mm ²)
แขนหมุน (Arm)	$L_{CoM1} = 35.402 \text{ mm}$	$I_{CoM1} = 769047.65 \text{ g} \cdot \text{mm}^2$
ลูกตุ้ม (Pendulum)	$L_{CoM2} = 55.75 \text{ mm}$	$I_{CoM2} = 130015.24 \text{ g} \cdot \text{mm}^2$

Kinematics

การวิเคราะห์จลนศาสตร์ (Kinematics) และพลวัต (Dynamics) ของระบบ Rotary Inverted Pendulum (RIP) ถือเป็นรากฐานสำคัญของโครงการนี้ โดยจลนศาสตร์ถูกใช้หาตำแหน่งและความเร็วของจุดศูนย์กลางมวล (CoM) ของแขนหมุนและลูกตุ้มใน World Frame โดยการคำนวณเหล่านี้ต้องอ้างอิงถึงระยะ CoM ที่ถูกต้องจากจุดหมุนจริง โดยนำไปสร้างฟังก์ชัน Lagrangian เพื่อให้ได้สมการพลวัตที่ไม่เป็นเชิงเส้น (Nonlinear EOMs) ซึ่งสมการเหล่านี้เป็นแบบจำลองทางคณิตศาสตร์ที่รวมผลกระทบของมวล (M), แรง Coriolis/Centripetal (C) และแรงโน้มถ่วง (G) ไว้ด้วยกัน ซึ่งเป็นแบบจำลองที่แม่นยำสำหรับการ Linearization รอบจุดตั้งตรง เพื่อออกแบบตัวควบคุม LQR และเป็นพื้นฐานในการแสดงผลการเคลื่อนไหวแบบ Real-time ใน RViz

1. Forward Pose Kinematics

1.1 Modified Denavit-Hartenberg (MDH Parameters)



รูปที่ 5 การตั้ง Frame สำหรับการหา MDH Parameters

MDH Parameters คือชุดตัวเลขที่ใช้กำหนดจลนศาสตร์ของหุ่นยนต์แบบ Open Chain อย่างเป็นระบบ โดยการกำหนดเมทริกซ์การแปลงเอกพันธ์ระหว่าง Frame พิกัดของแต่ละข้อต่อ ช่วยให้สามารถอธิบายตำแหน่ง (Position) และทิศทาง (Orientation) ของปลายแขนกลได้อย่างถูกต้องในรูปของตัวแปรข้อต่อ

ในการวิเคราะห์จลนศาสตร์และออกแบบระบบควบคุมนั้น เช่น การคำนวณหาความเร็วเชิงมุมของข้อต่อ (Joint Velocity) และ ความเร็วของปลายแขนกล (End-effector Velocity) จำเป็นต้องเริ่มต้นจากการสร้าง Transformation Matrix (T) ที่ถูกต้อง โดย Transformation Matrix เป็นเมทริกซ์ขนาด 4×4 ที่ทำหน้าที่เป็นตัวแทนของ Forward Position Kinematics (FPK) โดยเมทริกซ์นี้ถูกสร้างขึ้นจากฟังก์ชันที่ขึ้นอยู่กับตัวแปรข้อต่อ (q) และ MDH Parameters จึงทำให้สามารถทราบ Pose ของทุกส่วนในหุ่นยนต์ ณ มุมข้อต่อใด ๆ ได้

ตารางแสดง MDH Parameters

i - 1	i	a	α	d	θ
0	1	0	0	L_0	q_1
1	2	L_1	90°	0	90°
2	3	0	90°	0	q_2
3	e	L_2	0	0	0
เมื่อ $L_0 = 0.2375$ m, $L_1 = 0.1332$ m และ $L_2 = 0.1350$ m					

การทราบ T อย่างถูกต้องจะนำไปสู่การคำนวณ Inverse Jacobian Matrix ($J(q)^{-1}$) โดยในที่นี้จำเป็นต้องคำนวณหา 0_1T , 0_2T , 0_3T และ 0_eT เพื่อการวิเคราะห์ความเร็วข้อต่อต่อไป

$${}^0_1T = \begin{bmatrix} \cos q_1 & -\sin q_1 & 0 & 0 \\ \sin q_1 & \cos q_1 & 0 & 0 \\ 0 & 0 & 1 & L_0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.2220 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^0_2T = \begin{bmatrix} 0 & -\cos q_1 & \sin q_1 & L_1 \cos q_1 \\ 0 & -\sin q_1 & -\cos q_1 & L_1 \sin q_1 \\ 0 & 0 & 1 & L_0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 & 0.1330 \\ 0 & 0 & -1 & 0 \\ 1 & 0 & 0 & 0.2220 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^0_3T = \begin{bmatrix} \sin q_2 \sin q_1 & \cos q_2 \sin q_1 & \cos q_1 & L_1 \cos q_1 \\ -\sin q_2 \cos q_1 & -\cos q_2 \cos q_1 & \sin q_1 & L_1 \sin q_1 \\ \cos q_2 & -\sin q_1 & 0 & L_0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0.1330 \\ 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0.2220 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$${}^0_eT = \begin{bmatrix} \sin q_2 \sin q_1 & \cos q_2 \sin q_1 & \cos q_1 & L_1 \cos q_1 - L_2 \sin q_2 \sin q_1 \\ -\sin q_2 \cos q_1 & -\cos q_2 \cos q_1 & \sin q_1 & L_1 \sin q_1 + L_2 \sin q_2 \cos q_1 \\ \cos q_2 & -\sin q_1 & 0 & L_0 + L_2 \cos q_2 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0.1330 \\ 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0.3570 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

1.2 Geometric Approach

Geometric Approach เป็นวิธีการวิเคราะห์จลนศาสตร์ที่อาศัยหลักตรีโกณมิติ (Trigonometry) เพื่อหาความสัมพันธ์เชิงตำแหน่งของลิงก์และข้อต่อโดยตรงจากรูปร่างเรขาคณิตของหุ่นยนต์ วิธีนี้ช่วยให้สามารถตรวจสอบความถูกต้องของแบบจำลองได้อย่างเป็นรูปธรรม เนื่องจากไม่พึ่งพารูปแบบการกำหนดเฟรมใดเป็นพิเศษ

ในการพัฒนาแบบจำลองจลนศาสตร์ของระบบ การคำนวณด้วย Geometric Approach ถูกนำมาใช้เพื่อตรวจสอบความสอดคล้องของ MDH Parameters ที่สร้างเฟรมและเมทริกซ์การแปลงของระบบ หากผลลัพธ์จากทั้งสองวิธีให้ตำแหน่งและการเคลื่อนที่ที่ตรงกัน แสดงว่า MDH Parameters ที่ตั้งไว้มีความถูกต้องและสะท้อนเรขาคณิตทางกายภาพของหุ่นยนต์ได้อย่างเหมาะสม

$$\begin{bmatrix} P_{x1} \\ P_{y1} \\ P_{z1} \end{bmatrix} = \begin{bmatrix} L_1 \cos q_1 \\ L_1 \sin q_1 \\ L_0 \end{bmatrix}$$

$$\begin{bmatrix} P_{x2} \\ P_{y2} \\ P_{z2} \end{bmatrix} = \begin{bmatrix} L_1 \cos q_1 - L_2 \sin q_2 \sin q_1 \\ L_1 \sin q_1 + L_2 \sin q_2 \cos q_1 \\ L_0 + L_2 \cos q_2 \end{bmatrix}$$

2. Forward Velocity Kinematics

Forward Velocity Kinematics (FVK) เป็นการวิเคราะห์ที่ต่อเนื่องจาก Forward Position Kinematics (FPK) โดยมีเป้าหมายคือการหาความเร็วเชิงเส้น (Linear Velocity (v)) และ ความเร็วเชิงมุม (Angular Velocity (ω)) ของปลายแขนกล (End-effector) หรือจุดสนใจใด ๆ บนหุ่นยนต์

$$\begin{bmatrix} V_{x1} \\ V_{y1} \\ V_{z1} \end{bmatrix} = \begin{bmatrix} -\dot{q}_1 L_1 \sin q_1 \\ \dot{q}_1 L_1 \cos q_1 \\ 0 \end{bmatrix}$$

$$V_{ee} = \begin{bmatrix} V_{x2} \\ V_{y2} \\ V_{z2} \end{bmatrix} = \begin{bmatrix} -\dot{q}_1 L_1 \sin q_1 - L_2 (\dot{q}_1 \cos q_1 \sin q_2 + \dot{q}_2 \sin q_1 \cos q_2) \\ \dot{q}_1 L_1 \cos q_1 - L_2 (-\dot{q}_1 \sin q_1 \sin q_2 + \dot{q}_2 \cos q_1 \cos q_2) \\ -\dot{q}_2 L_2 \sin q_2 \end{bmatrix}$$

3. Inverse Jacobian Matrix

Jacobian Matrix เป็นเมทริกซ์ที่เป็นตัวเชื่อมโยงระหว่างความเร็วเชิงมุมของข้อต่อกับความเร็วจังหวัดของปลายแขนกล โดย Inverse Jacobian Matrix ถูกใช้เพื่อแปลงความเร็วของปลายแขนกลที่ต้องการ กลับไปเป็นความเร็วข้อต่อ ($\dot{q}$)

$$J(e) = \begin{bmatrix} \hat{Z}_i \times ({}^0P - {}^0P_i) \\ \hat{Z}_i \end{bmatrix}$$

ในระบบ Rotary Inverted Pendulum (RIP) ที่ถูกสร้างแบบจำลองด้วย MDH และมีโครงสร้างแบบ 3 Joints จำเป็นต้องใช้หลักการ Reduced Jacobian เพื่อให้การคำนวณความเร็วของปลายแขนกล (End-effector Velocity) และพลวัตสอดคล้องกับองศาอิสระที่ถูกขับเคลื่อนจริง (Actuated DOF)

$$J(e) = \begin{bmatrix} -L_1 \sin q_1 - L_2 \sin q_2 \cos q_1 & L_2 \sin q_1 \cos q_1 \\ L_1 \cos q_1 - L_2 \sin q_1 \sin q_2 & -L_2 \cos q_1 \cos q_2 \\ 0 & -L_2 \sin q_2 \\ 0 & \cos q_2 \\ 0 & \sin q_1 \\ 1 & 0 \end{bmatrix}$$

Jacobian Matrix เป็นเมทริกซ์ที่เป็นตัวเชื่อมโยงระหว่าง ความเร็วเชิงมุมของข้อต่อกับความเร็วเชิงพื้นที่ของปลายแขนกล โดย Inverse Jacobian Matrix ถูกใช้เพื่อแปลงความเร็วของปลายแขนกลที่ต้องการ กลับไปเป็นความเร็วข้อต่อ

$$\dot{q} = J(e)^{-1}V_{ee}$$

4. Dynamics Equation

สมการพลวัต (Dynamic Equations) คือชุดสมการเชิงอนุพันธ์อันดับสอง ที่อธิบายความสัมพันธ์ระหว่างแรงที่กระทำต่อระบบ (Generalized Forces/Torques) กับความเร่ง (Accelerations) ที่เกิดขึ้นในระบบกลไกที่มีหลายองศาอิสระ สำหรับ RIP จะมีการใช้ Lagrange Method ในรูปแบบเมทริกซ์มาตรฐานของระบบหุ่นยนต์ ดังนี้

4.1 Lagrange Method

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q)$$

ในการสร้างสมการพลวัตของระบบ RIP จะนิยามจากฟังก์ชัน Lagrangian ที่แสดงผลต่างระหว่างพลังงานจลน์รวม (T) และพลังงานศักย์รวม (V)

$$L = T - V$$

พลังงานจลน์จะถูกคำนวณจากการใช้ Forward Velocity Kinematics ที่ได้จากการหาอนุพันธ์เทียบกับเวลาของพิกัด CoM ที่ถูกต้อง ซึ่งรวมผลกระทบของมวล Link 1 และ Link 2 และโมเมนต์ความเฉื่อยรวม ส่วนพลังงานศักย์จะขึ้นอยู่กับพิกัดแนวตั้งของลูกตุ้ม

$$T = \frac{1}{2}m_1(V_{x2}^2 + V_{y2}^2 + V_{z2}^2) + \frac{1}{2}I_1\dot{q}_1 + \frac{1}{2}m_1(V_{x1}^2 + V_{y1}^2 + V_{z1}^2) + \frac{1}{2}I_2\dot{q}_2$$

$$V = m_2I_2 \cos q_2$$

4.2 Euler-Lagrange

$$\begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix} = \begin{bmatrix} \frac{d}{dt} \left(\frac{dL}{dq_1} \right) - \left(\frac{dL}{dq_1} \right) \\ \frac{d}{dt} \left(\frac{dL}{dq_2} \right) - \left(\frac{dL}{dq_2} \right) \end{bmatrix}$$

4.3 Dynamic Equations

$$M(q) = \begin{bmatrix} m_1 L_{CoM1}^2 - m_2 L_{CoM2}^2 \cos^2 q_2 + m_2 L_{CoM2}^2 + m_2 L_1^2 + I_{CoM1} & m_2 L_{CoM2} L_1 \cos q_2 \\ m_2 L_{CoM2} L_1 \cos q_2 & m_2 L_{CoM2}^2 + I_{CoM2} \end{bmatrix}$$

$$C(q, \dot{q}) = \begin{bmatrix} L_{CoM2} \dot{q}_2 m_2 (L_{CoM2} \dot{q}_1 \sin 2q_2 - L_1 \dot{q}_2 \sin q_2) \\ -\frac{1}{2} L_{CoM2} m_2 (L_{CoM2} \sin 2q_2) \dot{q}_1^2 \end{bmatrix}$$

$$G(q) = \begin{bmatrix} 0 \\ -L_{CoM2} m_2 g \sin q_2 \end{bmatrix}$$

ค่า

$$L_1 = 133 \text{ mm}$$

$$L_2 = 135 \text{ mm}$$

$$L_{CoM1} = 35.402 \text{ mm}$$

$$L_{CoM2} = 55.75 \text{ mm}$$

$$I_{CoM1} = 769047.65 \text{ g} \cdot \text{mm}^2$$

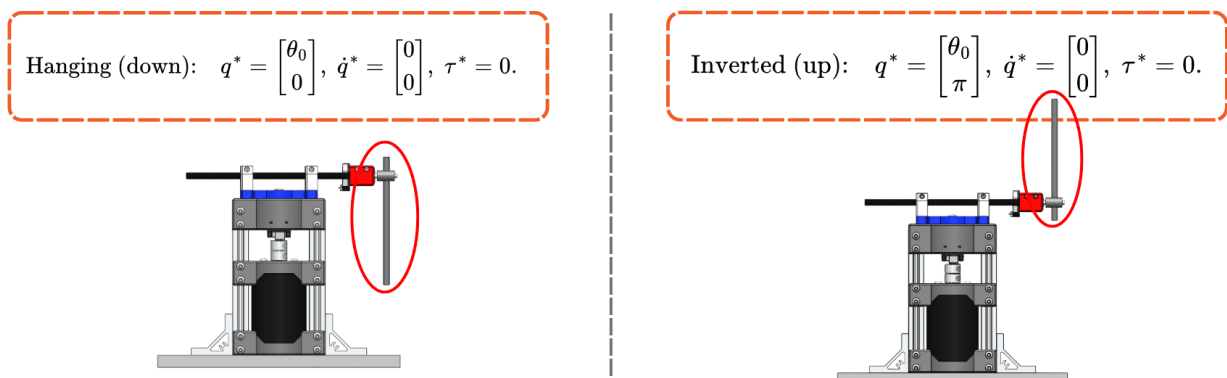
$$I_{CoM2} = 130015.24 \text{ g} \cdot \text{mm}^2$$

State Space ของระบบ

จากสมการ Dynamic ของ Rotary Inverted Pendulum พบว่ามีความซับซ้อนและความเป็น Non Linear ซึ่งปรากฏอยู่ในสมการในรูปของฟังก์ชันตรีโกณมิติและพจน์ยกกำลังสอง ซึ่งทำให้ไม่สามารถใช้เทคนิคการควบคุมแบบเชิงเส้นได้โดยตรง เนื่องจากสมการที่เกี่ยวข้องกับมุมและความเร็วไม่สามารถแสดงเป็นรูปเชิงเส้นในลักษณะทั่วไปได้ ดังนั้น เพื่อให้สามารถใช้เทคนิคการควบคุมที่เป็นเชิงเส้นได้ จำเป็นต้องดำเนินการ Linearization ก่อน

กระบวนการ Linearization คือการประมาณพฤติกรรมของระบบไม่เชิงเส้นให้อยู่ในรูปเชิงเส้นรอบ ๆ จุดสมดุล (Equilibrium Point) ที่สนใจ โดยจะช่วยให้ระบบที่ไม่เชิงเส้นกลายเป็นระบบเชิงเส้นที่สามารถนำไปใช้ในการวิเคราะห์และออกแบบตัวควบคุมได้ง่ายขึ้น

ในกรณีของ Rotary Inverted Pendulum จุดสมดุลที่เราพิจารณามีอยู่ด้วยกัน 2 จุดหลัก ได้แก่:



รูปที่ 6 Balanced Point of Rotary Inverted Pendulum

1. จุดที่ลูกตุ้มห้อยลง (Downward Position) โดยจะมีมุมเป็น 0 องศา ไม่มีความเร็วและไม่มี Torque Input
2. จุดที่ลูกตุ้มตั้งตรง (Upright Position) โดยจะมีมุมเป็น 180 องศา ไม่มีความเร็วและไม่มี Torque Input

การทำ Linearization ได้เลือกจุดที่ลูกตุ้มตั้งตรงเนื่องจากเป้าหมายหลักของการควบคุมในระบบ Rotary Inverted Pendulum ช่วยให้ระบบสามารถนำไปใช้ในเทคนิคการควบคุมที่เป็นเชิงเส้น ในกรณีนี้คือ Linear Quadratic Regulator ซึ่งจะช่วยออกแบบตัวควบคุมที่สามารถรักษาลูกตุ้มให้อยู่ในตำแหน่งตั้งตรงได้อย่างมีประสิทธิภาพ โดยไม่จำเป็นต้องคำนึงถึงพฤติกรรมที่ไม่เชิงเส้นทั้งหมดของระบบ

จากสมการ Dynamic ของระบบ สามารถกำหนด State Variable ของระบบและ State Derivative ของระบบได้ดังนี้

- Dynamic Equation $H(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = S \tau$
โดยที่ Matrix S เป็นการกำหนด Input ของระบบ $S = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$
- State Variable $x = \begin{bmatrix} q \\ \dot{q} \end{bmatrix}$

- State Derivative $\dot{x} = \begin{bmatrix} \dot{q} \\ \dot{q} \end{bmatrix} = \begin{bmatrix} \dot{q} \\ H(q)^{-1}(S\tau - C(q, \dot{q})\dot{q}) - G(q) \end{bmatrix}$

ทำการจัดรูป State Derivative ใหม่ให้อยู่ในรูปของ State Vector และ Input Vector จะได้ $\dot{x} = f(x, u)$ จากนั้นนำไปหาค่า Jacobian ของสมการเพื่อหาค่า System Matrix (A) และ Input Matrix (B) ดังนี้

$$A_\tau = \begin{bmatrix} 0 & 1 \\ H(q^*)^{-1} \frac{\delta G}{\delta q} \Big|_{q^*} & -H(q^*)^{-1} C(q^*, 0) \end{bmatrix} \quad B_\tau = \begin{bmatrix} 0 \\ H(q^*)^{-1} S \end{bmatrix}$$

โดยแทนค่า q^* ซึ่งเป็นจุดสมดุลที่เลือกมาในระบบ ทำให้พจน์ที่เป็น Non-Linear หายไปและระบบกลายเป็น Linear ในรอบ ๆ จุดสมดุลนั้น จากนั้นเพิ่ม Motor Dynamics เข้าไปในสมการ โดยการเปลี่ยนจาก Torque Input เป็น Voltage Input ตามลักษณะของ DC Motor ซึ่งสามารถหาความสัมพันธ์ระหว่าง Torque และ Voltage ได้ดังนี้

- ความสัมพันธ์ระหว่าง Torque และ Current $\tau = K_t i_m$

- สมการสำหรับ Motor Voltage $V_m = R_m i_m + K_m \dot{\theta}$

ทำการจัดรูปใหม่ จะได้ว่า $\tau = \frac{K_t}{R_m} (V_m - K_m \dot{\theta})$

เมื่อนำความสัมพันธ์ระหว่าง Torque และ Voltage ที่ได้มาแทนในสมการ State-Space จะได้ดังนี้

$$\begin{aligned} \dot{x} &= A_\tau x + B_\tau \\ \dot{x} &= A_\tau x + B_\tau \left(\frac{K_t}{R_m} V_m - \frac{K_t K_m}{R_m} \dot{\theta} \right) \\ \dot{x} &= \left(A_\tau - B_\tau \frac{K_t K_m}{R_m} e_\theta^\top \right) x + B_\tau \frac{K_t}{R_m} V_m \end{aligned}$$

และจะหา System Matrix (A) และ Input Matrix (B) ใหม่ได้ดังนี้

$$A \rightarrow A_\tau - \begin{bmatrix} 0 & 0 & \frac{K_t K_m}{R_m} & B & 0 \end{bmatrix} \quad B \rightarrow \frac{K_t}{R_m} B_\tau$$

จากทั้งหมดที่กล่าวมานั้น สามารถสรุป State-Space Model ของระบบได้ดังนี้

- State Vector $x = \begin{bmatrix} \theta \\ \alpha \\ \dot{\theta} \end{bmatrix}$ และ Output Vector $y = \begin{bmatrix} \theta \\ \alpha \end{bmatrix}$

- System Matrix $A = \frac{1}{J_T} \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{1}{4} m_p L_p^2 L_r g & -\left(J_p + \frac{1}{4} m_p L_p^2\right) B_r & -\frac{1}{2} m_p L_p L_r B_p \\ 0 & \frac{1}{2} m_p L_p g (J_r + m_p L_r^2) & \frac{1}{2} m_p L_p L_r B_r & (J_r + m_p L_r^2) B_p \end{bmatrix}$

- Input Matrix $B = \frac{1}{J_T} \begin{bmatrix} 0 \\ 0 \\ J_p + \frac{1}{4}m_p L_p^2 \\ \frac{1}{4}m_p L_p L_r \end{bmatrix}$
- Output Matrix $C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
- Feed Forward Matrix $D = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

Energy-Based Swingup

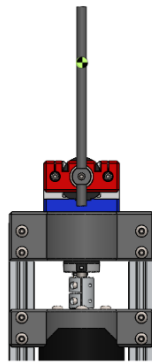
Energy-Based Swingup คือเทคนิคที่ใช้พลังงานในการควบคุมการเคลื่อนที่ของระบบ โดยมุ่งเน้นการเพิ่มพลังงานในระบบเพื่อให้สามารถยกลูกตุ้มขึ้นไปในตำแหน่งที่ตั้งตรง ซึ่งสามารถอธิบายได้ดังนี้

1. Reference Energy

Reference Energy คือพลังงานที่ต้องการในระบบเมื่ออยู่ในตำแหน่งที่ตั้งตรงของลูกตุ้ม (Upright Position) โดยจะคำนวณได้จากสมการ

$$E_{\text{ref}} = 2 M_p g L_p$$

โดยที่ M_p คือ มวลของลูกตุ้ม
 g คือ ความเร่งเนื่องจากแรงโน้มถ่วง
 L_p คือ ความยาวของแขนของลูกตุ้ม



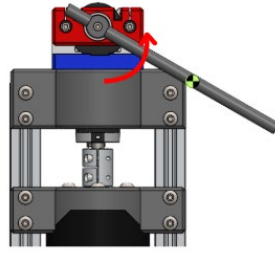
รูปที่ 7 Upright Position Pendulum

2. Energy of the System

Energy of the System ประกอบด้วยพลังงานจลน์และพลังงานศักย์ สามารถคำนวณได้จากสมการ

$$E = \frac{1}{2} J_p \dot{\alpha}^2 + M_p g L_p (1 - \cos \alpha)$$

โดยที่ J_p คือ Moment of Inertia ของ Pendulum
 $\dot{\alpha}$ คือ ความเร็วเชิงมุมของลูกตุ้ม
 α คือ มุมของลูกตุ้มจากตำแหน่งที่ตั้งตรง



รูปที่ 8 Pendulum ที่มุมต่าง ๆ

3. การเปลี่ยนแปลงของพลังงานเป็นความแตกต่างระหว่างพลังงานของระบบและพลังงานอ้างอิง

$$\Delta E = E - E_{\text{ref}}$$

4. การออกแบบการควบคุมจะใช้การกระตุ้นพลังงาน โดยใช้พลังงานที่เปลี่ยนแปลงไปเป็นตัวขับเคลื่อนการควบคุมที่สัมพันธ์กับความเร็วเชิงมุมและมุมของลูกตุ้ม โดยสมการของการควบคุมจะมีลักษณะดังนี้

$$U = k_E \Delta E \propto \cos \alpha$$

โดยที่ U คือ แรงขับเคลื่อนที่เรากำหนดให้กับระบบ

k_E คือ ค่าคงที่ในการปรับขนาดการควบคุม

เมื่อนำไปเขียนโปรแกรมจะสามารถเขียนได้ดังนี้

```
float EnergyCtrl_Update (EnergyCtrl *ec, float alpha, float alpha_dot) {
    ec->E = 0.5f * ec->Jp * alpha_dot * alpha_dot
        + ec->Mp * ec->g * ec->Lp * (1.0f - cosf(alpha));

    ec->dE = ec->E - ec->Eref;

    ec->ap = ec->kE * ec->dE * alpha_dot * cosf(alpha);

    ec->ap = sat_smooth(ec->ap, 1.0f);

    return ec->ap;
}
```

รูปที่ 9 โปรแกรมการควบคุม Energy-Based Swingup

Linear Quadratic Regulator

LQR เป็น Full State Feedback Control ประเภทหนึ่งที่ใช้ในการควบคุมระบบเชิงเส้น ซึ่งจะทำการคูณค่าของ State Vector กับ LQR Gain เพื่อควบคุมพฤติกรรมของระบบโดยการลด Cost Function โดยในกรณีของระบบ Discrete-time การออกแบบ LQR จะใช้ Discrete-time State Space Model โดยที่ต้องทำการเลือก Q และ R สำหรับ Cost Function ในการควบคุม

Cost Function ของ LQR คือการรวมค่าของ State และ Control Input ตลอดเวลาตั้งแต่ $k = 0, 1, 2, \dots$ ดังนี้

$$J = \sum_{k=0}^{\infty} (x_k^T Q x_k + u_k^T R u_k)$$

โดยที่ x_k คือ State Vector ของระบบในเวลา k
 u_k คือ Control Input หรือ Input Vector ในเวลา kk
 Q คือ Weighting Matrix สำหรับกำหนดความสำคัญในการลด Error ของ State
 R คือ Weighting Matrix สำหรับควบคุม Control Input

วิธีคำนวณ LQR Gain จะใช้ Discrete Riccati Equation

$$P = A_d^T P A_d - A_d^T P B_d (R + B_d^T P B_d)^{-1} B_d^T P A_d + Q$$

โดยที่ A_d คือ Discrete-time System Matrix
 B_d คือ Discrete-time Input Matrix
 P คือ Solution Matrix ของสมการ Riccati ที่ใช้ในการคำนวณ Gain

เมื่อหาค่า Solution Matrix ได้แล้วจะนำไปทำการคิดค่า Gain ด้วยสมการ ดังนี้

$$K = (R + B_d^T P B_d)^{-1} B_d^T P A_d$$

โดยกระบวนการนี้สามารถทำได้ด้วยคำสั่ง `dlqr()` ใน Matlab ต่อมาจึงเลือกค่า Q และ R สำหรับระบบ โดยใช้ Bryson's Rule ซึ่งเป็นเทคนิคที่ใช้ในการตั้ง Q และ R ตามค่า Maximum และ Minimum ที่เราต้องการให้เกิดในระบบ โดยได้ทำการกำหนดค่าต่าง ๆ ดังนี้

```

%Bryson's rule
theta_max = 0.10; % rad
alpha_max = 0.10; % rad
thetaDot_max = 1.0; % rad/s
alphaDot_max = 1.0; % rad/s
u_max = 24.0; % V

Qb = diag([1/theta_max^2, 1/alpha_max^2, 1/thetaDot_max^2, 1/alphaDot_max^2]);
Rb = 1/u_max^2;

w_alpha = 3.0; w_thetadot = 0.5;
Q = Qb .* diag([1, w_alpha, w_thetadot, 1]);
R = Rb;

```

รูปที่ 10 การกำหนดค่าต่าง ๆ ใน Matlab

จากนั้นทำการนำค่า Gain ที่ได้จาก Matlab มาใส่ลงไปในโปรแกรม และทำการเขียน Full State Feedback ใน Microcontroller ดังนี้

```

void LQR_SetState (LQR_Controller *ctrl, float32_t theta, float32_t alpha,
                  float32_t theta_dot, float32_t alpha_dot) {
    ctrl->x_data[0] = theta;
    ctrl->x_data[1] = alpha;
    ctrl->x_data[2] = theta_dot;
    ctrl->x_data[3] = alpha_dot;
}

float32_t LQR_Update (LQR_Controller *ctrl) {
    arm_mat_mult_f32(&ctrl->K, &ctrl->x, &ctrl->u);
    ctrl->u_data[0] = -ctrl->u_data[0];

    if (ctrl->u_data[0] > ctrl->u_limit)
        ctrl->u_data[0] = ctrl->u_limit;
    else if (ctrl->u_data[0] < -ctrl->u_limit)
        ctrl->u_data[0] = -ctrl->u_limit;

    return ctrl->u_data[0];
}

```

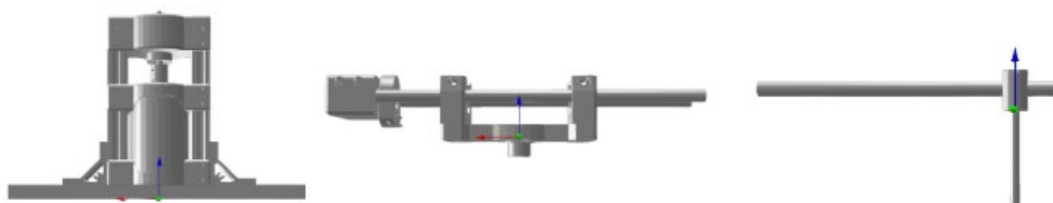
รูปที่ 11 โปรแกรมการควบคุม LQR

Modelling (URDF)

การสร้างแบบจำลอง URDF เพื่อนำไปใช้ร่วมกับ ROS2 จำเป็นต้องเตรียมโมเดลใน SolidWorks ให้ถูกต้องตามโครงสร้างลำดับ Link และ Joint ของระบบ Rotary Inverted Pendulum (RIP) เพื่อให้ขั้นตอนการกำหนด Coordinate Frame และการ Export URDF สามารถทำได้ถูกต้องและสอดคล้องกับ Hardware จริง โดยขั้นตอนหลักในการเตรียมโมเดลใน SolidWorks มีดังนี้

1. กำหนดโครงสร้างย่อยของระบบ (Sub-Assembly Structure)

โมเดลของ RIP ถูกแบ่งออกเป็น 3 ส่วนหลัก เพื่อให้ตรงกับ Link ของ URDF ได้แก่



รูปที่ 12 Base Assembly, Arm Assembly และ Pendulum Assembly

- Base Assembly
 - ประกอบด้วยอะลูมิเนียมโปรไฟล์ ฐานยึดมอเตอร์ และเพลายึดแขน
 - ทำหน้าที่เป็น Root Link (Base Link) ใน URDF
- Arm Assembly
 - ประกอบด้วย Base Swing Arm, Bearing Holder, Ball Bearing, Carbon Fiber Arm Shaft และ Shaft Support Block
 - ทำหน้าที่เป็น Arm ใน URDF
- Pendulum Assembly
 - ประกอบด้วย Pendulum Rotating Shaft และ Pendulum Shaft
 - ทำหน้าที่เป็น Pendulum ใน URDF

2. ตั้งค่า Coordinate Frame ของแต่ละ Link

การสร้าง URDF ต้องมีการกำหนดจุดอ้างอิง (Reference Frame) และแกนหมุนที่สอดคล้องกับ Hardware จริง โดยใน SolidWorks จะใช้เครื่องมือ Reference Geometry -> Coordinate System เพื่อสร้างเฟรมพิกัดของแต่ละ Link ตามลำดับ Kinematic Chain ได้แก่ base_frame, arm_frame และ pendulum_frame ซึ่งมีหลักการกำหนดเฟรมพิกัดดังนี้

- ต้นกำเนิดเฟรม (Origin) ต้องตรงกับตำแหน่งจุดหมุนของ Joint เช่น
 - เฟรมของ arm_link ต้องเริ่มที่แกนหมุนของมอเตอร์
 - เฟรมของ pendulum_link ต้องเริ่มที่ตำแหน่งแกนหมุนของลูกตุ้ม
- แนวแกนหมุน (Axis Naming) ควรตั้งให้สอดคล้องกับ ROS Convention (REP-103)
 - Z-axis = แกนหมุนของ Joint (Revolute Joint)
 - X-axis = Pointing Forward
 - Y-axis = Pointing Left

3. กำหนดชนิด Joint ระหว่าง Link

สำหรับระบบ RIP ได้กำหนดชนิด Joint ดังนี้

- Base -> Arm: ใช้ Revolute Joint กำหนดแกนหมุนตามแกนมอเตอร์
- Arm -> Pendulum: ใช้ Revolute Joint กำหนดแกนหมุนตรงกับ Pendulum Rotating Shaft

4. Export URDF ผ่าน SolidWorks URDF Exporter

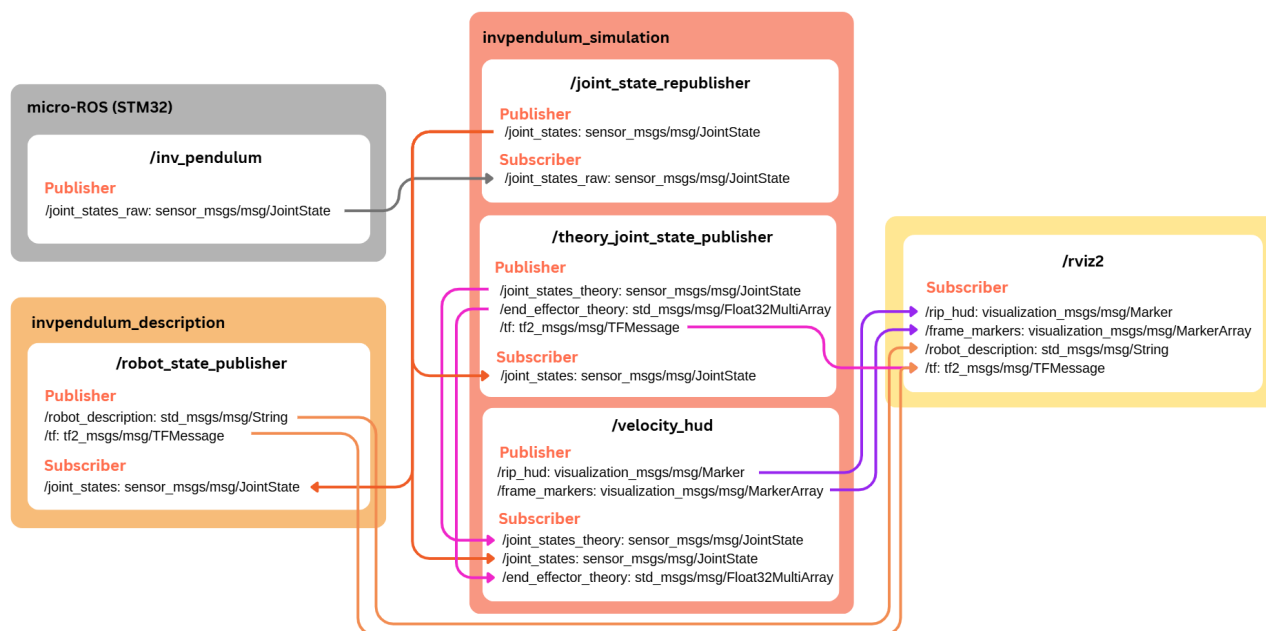
เมื่อเตรียมโครงสร้าง Assembly, Mates และ Coordinate Frames เรียบร้อยแล้ว จึงใช้ Plug-in ในโปรแกรม SolidWorks -> Tools -> Robot Export -> URDF โดยกำหนด Link, ชนิด Joint ตาม Mates, แกนหมุนของ Joint, เฟรมพิกัดที่สร้างไว้ รวมถึง Collision and Visual สุดท้ายแล้วผลลัพธ์ที่ได้ คือ ไฟล์ model.urdf และ mesh (.STL หรือ .DAE) ที่พร้อมใช้กับ robot_state_publisher และ RViz2

5. กำหนด Frame Transformations และโครงสร้างของระบบ

การกำหนดตำแหน่งและทิศทางของแต่ละ Joint และ Link ผ่านการแปลง Frame ที่ได้วิเคราะห์ไว้ เป็นข้อมูลพื้นฐานที่จำเป็นต้องนำไปเขียนในไฟล์ URDF ให้ถูกต้องตาม Kinematics เพื่อให้ ROS2 Node สามารถรับข้อมูล Joint States จาก Microcontroller และเปรียบเทียบกับเคลื่อนที่ที่เกิดจากการคำนวณทางทฤษฎีในลักษณะ Dynamic Motion ได้อย่างแม่นยำและ Real-time ผ่าน Rviz โดยจากระบบดังกล่าวจะสามารถกำหนดได้ดังนี้

- Base to Arm Joint (Rev_Arm) (Origin: xyz="0.0002 0 0.212") : Arm Joint สามารถหมุนได้สูงจาก Base 0.212 เมตร และมีแกนหมุนรอบแกน Z (แนวตั้ง) โดยส่วนนี้ถือเป็น Rotary ของระบบ
- Arm to Pendulum Joint (Rev_Pendulum) (Origin: xyz="0.1215 0 0.0255") : Arm Joint ห่างจาก Pendulum Joint 0.1215 เมตร และ Offset ในแนวตั้ง 0.0255 เมตร โดยในการแปลง Frame ต้องผ่านการหมุน rpy="0 1.5708 0" (หมุน 90° รอบแกน Y) การหมุนนี้แปลงแกน Z ให้ชี้ในแนวนอน ทำให้ลูกตุ้มแกว่งได้ถูกต้อง (แกน Z จะชี้ตั้งฉากกับแขนเสมอ)
- Pendulum Joint to End-effector (Origin: xyz="0.135 0 0.0115") : End-effector ห่างจาก Pendulum Joint 0.135 เมตร ทำให้ระยะนี้ถือเป็นระยะเอื้อมทั้งหมดของปลาย Pendulum

ROS Implementation



รูปที่ 13 System Architecture

แผนภาพนี้อธิบายสถาปัตยกรรมของระบบ Rotary Inverted Pendulum ที่ใช้เปรียบเทียบการเคลื่อนที่จริงจาก Hardware กับการตอบสนองจากแบบจำลองในทางทฤษฎี และแสดงผลร่วมกันใน RViz โดยแบ่งออกเป็น 4 ส่วนหลักดังนี้

1. Hardware Interface – micro-ROS (STM32)

ส่วนนี้ประกอบด้วยบอร์ด NUCLEO-G474RE ที่รัน micro-ROS และทำหน้าที่เป็น Node /inv_pendulum ซึ่งเป็นข้อมูลจริงที่ได้จากการทำงานของระบบ Hardware โดยบอร์ดจะอ่านค่ามุม (Position) และความเร็วเชิงมุม (Velocity) ของ Arm และ Pendulum ผ่าน Encoder เพื่อ Publish ข้อมูลออกไป และที่สำคัญยังเป็นส่วนที่รับผิดชอบการควบคุม Rotary Inverted Pendulum (RIP) ทั้งระบบ โดยใช้แนวคิด State-Space Control ประกอบด้วย 2 Mode หลัก ได้แก่ Swing-Up Control ที่ใช้พลังงานของระบบ (Energy-based Method) ในการเพิ่มโมเมนตัมให้ Pendulum แกว่งจนสามารถเข้าสู่บริเวณใกล้ตำแหน่งสมดุลกลับหัว (Upright Equilibrium) ได้ และ LQR Stabilization ที่เมื่อ Pendulum เข้าใกล้ตำแหน่งที่ตั้งตรง ระบบจะสลับไปใช้ Linear Quadratic Regulator (LQR) ซึ่งออกแบบจากแบบจำลองเชิงสถานะ ทำให้สามารถควบคุมให้ Pendulum รักษาสมดุลในตำแหน่งที่ตั้งตรงได้อย่างเสถียรและตอบสนองได้อย่างรวดเร็ว

Publisher:

- /joint_states_raw: sensor_msgs/msg/JointState

ส่งข้อมูลตำแหน่งและความเร็วเชิงมุมของ Arm และ Pendulum ที่อ่านจาก Encoder แบบ Real-time

2. Robot Description & TF – invpendulum_description

ส่วนนี้ประกอบด้วย Node /robot_state_publisher ซึ่งสร้างโมเดลจาก URDF และส่ง Transform (/tf) ทำให้ได้โมเดล 3 มิติของ Rotary Inverted Pendulum ที่ซึบตามมูมจริงจาก Hardware

Subscriber:

- /joint_states: sensor_msgs/msg/JointState
- รับข้อมูลมูมข้อต่อที่ผ่านการจัดเรียงมาอย่างถูกต้องจาก /joint_state_republisher

Publisher:

- /robot_description: std_msgs/msg/String
- ส่งข้อมูล URDF ของ Rotary Inverted Pendulum ให้ RViz ใช้ Download โมเดล 3 มิติ
- /tf: tf2_msgs/msg/TFMessage
- กระจาย Frame การเคลื่อนที่ของ Link ในระบบตามค่ามูมของ Joint States

3. Simulation & Comparison – invpendulum_simulation

ส่วนนี้ประกอบด้วย 3 Node หลักที่ทำหน้าที่ประมวลผลและเปรียบเทียบข้อมูลจริงกับข้อมูลทางทฤษฎี

3.1 /joint_state_republisher

Node นี้รับข้อมูลจริงจาก Hardware แล้วจัดรูปแบบให้ถูกต้องตาม URDF ก่อนกระจายต่อให้ Node อื่น

Subscriber:

- /joint_states_raw: sensor_msgs/msg/JointState

Publisher:

- /joint_states: sensor_msgs/msg/JointState

3.2 /theory_joint_state_publisher

Node นี้ใช้ข้อมูลจาก Hardware ในการคำนวณแบบจำลองเชิงทฤษฎีของระบบ โดยมีขั้นตอนดังนี้

- 1) คำนวณ Forward Kinematics (FK) จากข้อมูลมูมข้อต่อที่วัดได้จาก Hardware ด้วยแบบจำลอง MDH (DHRobot/RevoluteMDH) เพื่อระบุตำแหน่งและทิศทางของ Link รวมถึงใช้ในการคำนวณ Jacobian ของระบบที่ตำแหน่งปัจจุบัน
- 2) ใช้ Jacobian ร่วมกับความเร็วข้อต่อที่วัดได้จาก Hardware เพื่อคำนวณความเร็วเชิงเส้นและเชิงมุมของ End Effector ตามแบบจำลองทฤษฎี
- 3) จากนั้นใช้ Inverse Jacobian รับค่าของ End Effector ที่คำนวณได้ในข้อก่อนหน้า แล้วประมาณกลับเป็นความเร็วข้อต่อเชิงทฤษฎี ซึ่งเป็นความเร็วของข้อต่อทั้งสองที่สอดคล้องกับแบบจำลองของระบบ

Subscriber:

- /joint_states: sensor_msgs/msg/JointState

Publishers:

- /joint_states_theory: sensor_msgs/msg/JointState
ส่งข้อมูลตำแหน่งและความเร็วเชิงมุมของ Arm และ Pendulum ที่ได้จากการคำนวณทางทฤษฎี
- /end_effector_theory: std_msgs/msg/Float32MultiArray
ส่งข้อมูลขนาดของความเร็วเชิงเส้นและเชิงมุมของ End Effector ที่ได้จากการคำนวณทางทฤษฎี
- /tf: tf2_msgs/msg/TFMessage
กระจาย Frame การเคลื่อนที่ของ Link ตามค่ามุมที่คำนวณได้จากทฤษฎี

3.3 /velocity_hud

Node นี้เป็นตัวสร้างข้อมูลแสดงผล (Head-up Display) บน RViz เพื่อให้เห็นความต่างระหว่างข้อมูลจริงที่ได้จาก Hardware และข้อมูลที่ได้จากการคำนวณทางทฤษฎี

Subscribers:

- /joint_states: sensor_msgs/msg/JointState
- /joint_states_theory: sensor_msgs/msg/JointState
- /end_effector_theory: std_msgs/msg/Float32MultiArray

Publishers:

- /rip_hud: visualization_msgs/Marker
ส่งข้อความ HUD สรุปค่าความเร็วของข้อต่อทั้งจาก Hardware และทฤษฎี
- /frame_markers: visualization_msgs/MarkerArray
ส่งจุดสีที่แสดงตำแหน่ง Frame ของ Hardware และทฤษฎี เพื่อให้เห็น Frame ชัดเจนมากขึ้น

4. Visualization – RViz2

ส่วนนี้เป็นการแสดงผลโมเดล 3 มิติของ Rotary Inverted Pendulum ที่ขยับตามมุมจาก Hardware จริง โดยจะมี Frame ทฤษฎีปรากฏควบคู่กับ Frame จริง รวมทั้งมี HUD แสดงข้อมูลความเร็วและค่าการคำนวณต่าง ๆ

Subscribers:

- /robot_description: std_msgs/msg/String
- /tf: tf2_msgs/msg/TFMessage
- /rip_hud: visualization_msgs/Marker
- /frame_markers: visualization_msgs/MarkerArray

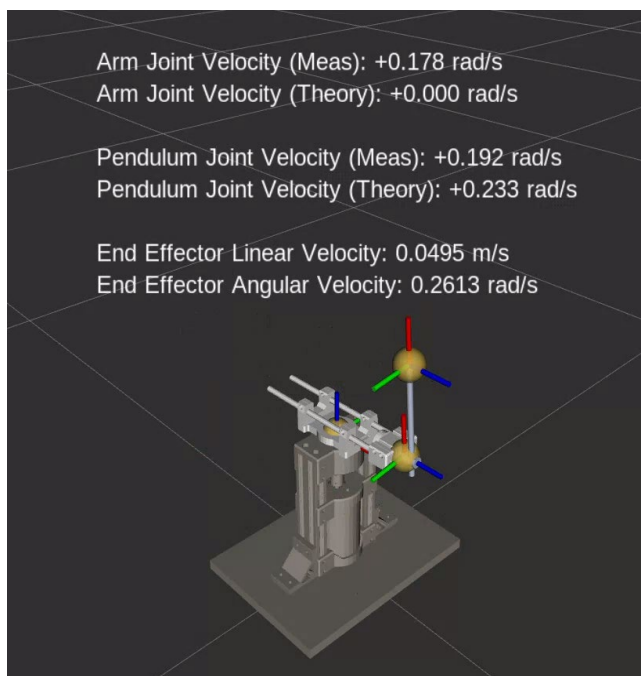
สรุปและวิเคราะห์ผลการทดลอง

ระบบ Rotary Inverted Pendulum (RIP) ที่พัฒนาในโปรเจกต์นี้สามารถทำงานได้อย่างถูกต้องตามที่ต้องการแบบไว้ โดยตัว Pendulum สามารถเคลื่อนที่ทั้งในโหมดการแกว่ง (Swing-up) และสามารถทรงตัวในตำแหน่งที่ตั้งตรงด้วยตัวควบคุมเชิงเส้นอย่าง LQR ได้อย่างมีประสิทธิภาพ การตอบสนองของระบบจริงสอดคล้องกับพฤติกรรมที่คาดหวังจากทฤษฎี กล่าวคือ เมื่อหมุน Arm สร้างแรงเหวี่ยง Pendulum สามารถถูกขับให้แกว่งและเข้าสู่ตำแหน่งที่ตั้งตรงได้ รวมถึงสามารถควบคุมเพื่อรักษาสถิตในจุดกำหนดได้อย่างต่อเนื่อง

นอกจากนี้ ระบบสามารถแสดงผลการทำงานแบบ Real-time ผ่าน RViz2 ได้อย่างถูกต้อง ประกอบไปด้วย

- โมเดล 3 มิติของ RIP ที่เคลื่อนที่ตามค่ามุมจริงจาก Hardware
- Frame ของ Link Arm, Pendulum และ End Effector จาก URDF (ข้อมูลจริง) และจากที่คำนวณได้จากแบบจำลองทฤษฎี
- HUD ข้อมูลความเร็วของ Arm และ Pendulum ทั้งจากค่าจริงและค่าที่ได้จากการคำนวณทฤษฎี
- ความเร็วเชิงเส้นและเชิงมุมของ End Effector ที่ปลาย Pendulum

แสดงให้เห็นว่าระบบการทำงานทั้งหมด ตั้งแต่การอ่านค่าจาก MCU การประมวลผล การคำนวณทางทฤษฎี จนถึงการแสดงผลทั้งหมด สามารถทำงานร่วมกันได้อย่างสมบูรณ์ ทำให้สามารถมองเห็นพฤติกรรมของระบบจริงเทียบกับแบบจำลองได้ตามที่ต้องการ



รูปที่ 14 ผลการทดลองของระบบ RIP ในรูปแบบ Hardware และ RViz

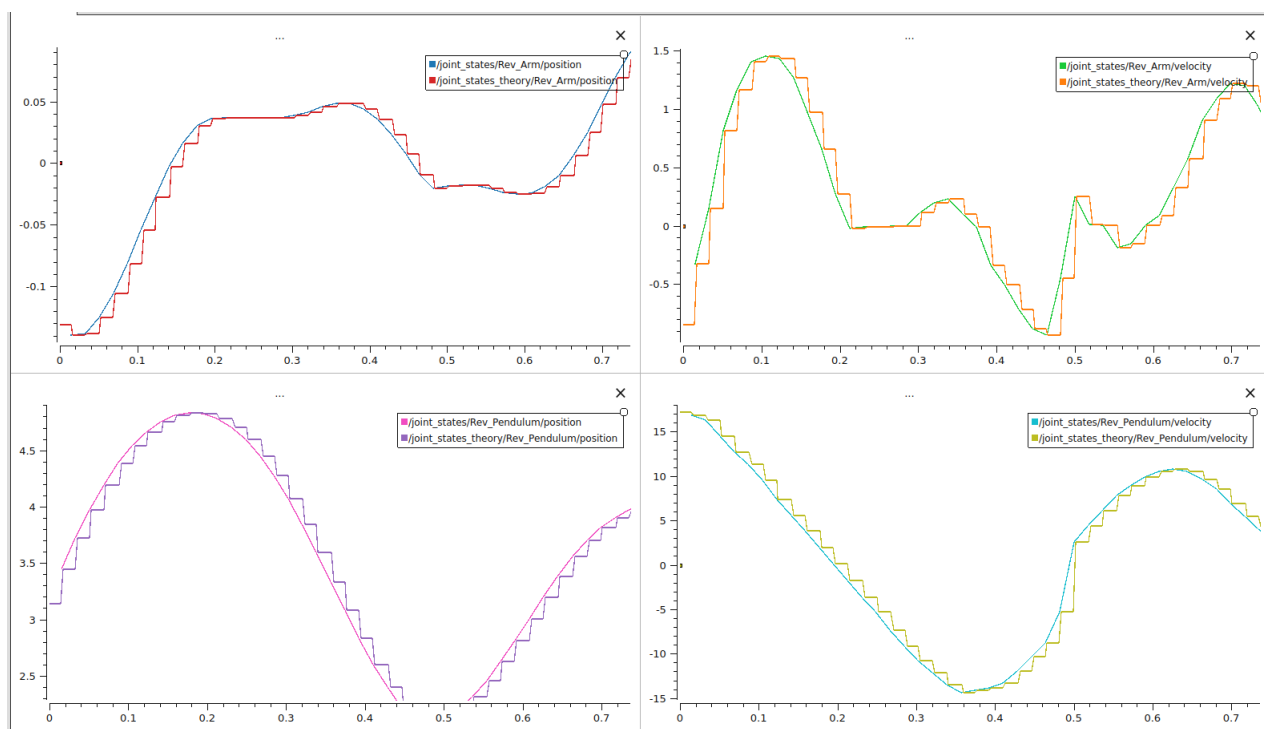
โดยเมื่อนำข้อมูลตำแหน่งและความเร็วของ Arm และ Pendulum ทั้งจากค่าจริงและค่าที่ได้จากการคำนวณทฤษฎี ไป Plot กราฟผ่าน Plotjuggler ทั้งหมด 4 กราฟ ได้แก่

- กราฟที่ 1 (ซ้ายบน) : ตำแหน่งของ Arm จากค่าจริงและค่าที่ได้จากการคำนวณทฤษฎี
- กราฟที่ 2 (ขวาบน) : ความเร็วของ Arm จากค่าจริงและค่าที่ได้จากการคำนวณทฤษฎี
- กราฟที่ 3 (ซ้ายล่าง) : ตำแหน่งของ Pendulum จากค่าจริงและค่าที่ได้จากการคำนวณทฤษฎี
- กราฟที่ 4 (ขวาล่าง) : ความเร็วของ Pendulum จากค่าจริงและค่าที่ได้จากการคำนวณทฤษฎี

จากกราฟตำแหน่งของ Arm และ Pendulum จะเห็นได้ว่าเส้นของ Real และ Theory มีค่าที่ตรงกันอย่างสมบูรณ์ อย่างไรก็ตาม เส้นกราฟของทฤษฎีมีลักษณะเป็นขั้นบันไดชัดเจนมากกว่า ซึ่งไม่ได้มาจากความคลาดเคลื่อนของแบบจำลอง แต่เกิดจากความแตกต่างของ Publish Rate ข้อมูลระหว่าง 2 แหล่งข้อมูล กล่าวคือ ข้อมูลจาก micro-ROS ถูกส่งผ่าน UART 115200 Baud ทำให้มีอัตราส่งสูงสุดประมาณ 55–57 Hz จึงได้ค่าใหม่จาก Encoder เพียงทุก 18 ms ในขณะที่ Node ทฤษฎี Publish ค่าผ่าน Timer ที่ 1000 Hz ส่งผลให้ค่าเดิมถูกทำซ้ำก่อนอัปเดตใหม่

สำหรับกราฟความเร็วของ Arm และ Pendulum พบว่าเส้นของ Real และ Theory มีความสอดคล้องกันอย่างใกล้เคียง ทั้งในแง่ของแนวโน้ม รูปร่างคลื่น จุดสูงสุด จุดต่ำสุด และจุดกลับทิศทาง โดยความเร็วทางทฤษฎีถูกคำนวณโดยใช้ความเร็วข้อต่อที่วัดได้จริงผ่าน Jacobian เพื่อหา End-Effector Velocity และใช้ Inverse Jacobian ขนาด 2×2 ในการประมาณกลับเป็นความเร็วข้อต่อเชิงทฤษฎี ซึ่งสอดคล้องกับองศาอิสระ (DOF) จริงของระบบ ในทางทฤษฎี หาก Jacobian ไม่อยู่ในสถานะที่ทำให้เกิด Singularity การอินเวอร์สจะทำให้ความเร็วทฤษฎีมีค่าใกล้เคียงกับค่าที่วัดได้จริงอย่างมาก ซึ่งก็ตรงกับผลที่พบในกราฟ ความต่างที่วัดได้มีค่าน้อยมาก ในระดับที่ต่ำกว่า 0.01 rad/s แสดงให้เห็นว่าการลดมิติของ Jacobian ให้ตรงกับระบบจริงช่วยให้ความเร็วทฤษฎีมีความถูกต้องสูงและสามารถอธิบายพฤติกรรมของระบบจริงได้อย่างมีประสิทธิภาพ ส่วนความแตกต่างด้านรูปร่าง อย่างลักษณะขั้นบันไดในเส้นของ Theory ก็เกิดขึ้นด้วยเหตุผลเดียวกับกราฟตำแหน่ง คือความถี่ในการ Publish ข้อมูลของ Node ทฤษฎีที่สูงกว่าสัญญาณจาก Encoder ทำให้ค่าความเร็วเชิงทฤษฎีถูก Publish ซ้ำในช่วงที่ยังไม่รับค่าใหม่จาก micro-ROS

ในภาพรวม กราฟทั้งสี่แสดงให้เห็นว่า Kinematics ของระบบ รวมถึง MDH และ Jacobian ที่ใช้ในระบบมีความถูกต้อง ทั้งตำแหน่งและความเร็วเชิงทฤษฎีสามารถติดตามพฤติกรรมของข้อมูลจริงได้อย่างใกล้เคียง โดยความแตกต่างส่วนใหญ่เกิดจากข้อจำกัดของอัตราการส่งข้อมูลมากกว่าจะเป็นความผิดพลาดของแบบจำลอง ถือเป็นหลักฐานว่าระบบทฤษฎีและข้อมูลจริงสามารถทำงานร่วมกันได้ถูกต้องและใช้วิเคราะห์พฤติกรรมของระบบ RIP ได้มีประสิทธิภาพ



รูปที่ 15 กราฟเปรียบเทียบตำแหน่งและความเร็วของ Arm และ Pendulum จากค่าจริงและทฤษฎี